UNIVERSITY OF ATHENS
DEPARTMENT OF INFORMATICS AND TELECOMMUNICATIONS

Reinforcement Learning And Stohastic Games Assignment

Student name: *Kleanthis Vasileiadis*
sdi: 1115202200017

Course: *Reinforcement Learning And Stohastic Games*
Semester: *Winter Semester 2025-2026*

Contents

1. Introduction

The goal of this project is to implement reinforcement learning algorithms for both single agent and multi agent enviroments, with a focus on actor critic methods. In particular, the project is divided into two main parts. The first part focuses on single agent problems under partial observability, while the second part examines a multi agent reinforcement learning algorithm based on the MADDPG paper.

For the single agent case, the environment is a Partially Observable Markov Decision Process (POMDP). In this setting, the agent does not have access to the full state of the environment and must rely on limited observations. To handle this, recurrent actor critic implementations are used, allowing the agent to use temporal information from past observations. Both LSTM based and GRU based actor critic models are implemented and compared in order to study their learning behavior and overall performance. In addition to the basic recurrent actor critic setup, several design choices are explored. Different hyperparameters are tested to examine their effect on training stability and results. Furthermore, an alternative network architecture is implemented, where a feed forward network performs feature extraction before the recurrent layer. This allows for a direct comparison between different architectural approaches for handling partial observability.

The second part of the project focuses on multi agent reinforcement learning. In this setting, multiple agents learn simultaneously, which introduces additional challenges due to the non stationary nature of the environment. To address this, the Multi Agent Deep Deterministic Policy Gradient (MADDPG) algorithm is implemented, following the approach described in the original paper. MADDPG extends actor critic methods by using a centralized critic during training, while each agent executes a decentralized policy based only on local observations.

2. Single Agent POMDPs

2.1. Environment Description and Partial Observability

The chosen environment for partially observable decision making is the `Blackjack-v1` environment from OpenAI Gym. This environment is considered partially observable because the agent does not have access to the full state of the game at any given time. Specifically, while the environment contains hidden information such as the dealer's face down card and the composition of the remaining deck, the agent's observation is limited to a summary of the current game situation.

The observation space of the Blackjack environment is a tuple of three components:

1. The player's current sum of cards (an integer between 4 and 21),
2. The dealer's visible card value (an integer between 1 and 10, where 1 represents an Ace),
3. A boolean indicating whether the player holds a usable Ace (an Ace counted as 11 without busting).

The partial observability arises because the agent cannot see the dealer's hidden card and cannot directly observe the probability distribution of future cards in the

deck. As a result, the agent must make decisions based on incomplete information, relying on the observed summary statistics and learned strategy to maximize expected rewards. Actions available to the agent are discrete: *hit* (draw another card) or *stand* (end the turn). The environment dynamics and reward structure are designed to emulate the real Blackjack game, providing positive reward for winning, negative reward for losing, and zero for a draw.

2.2. Recurrent Actor Critic Architectures

To address partial observability in the environment, a recurrent actor critic architecture is used. In this setup, both the policy(actor) and the value function(critic) are implemented using neural networks that include a recurrent component. The main idea is to allow the agent to have an internal memory of past observations, which helps make up for missing information at each time step.

The actor network is responsible for selecting actions based on the current observation and the hidden state of the recurrent layer. At each time step, the observation is first processed by a recurrent neural network, which can be either an LSTM or a GRU. The output of the recurrent layer captures temporal information from previous observations and is then passed through a feed forward network to produce the final action distribution(or action values, depending on the action space). This allows the policy to base its decisions on the current input and recent observations.

Similarly, the critic network uses a recurrent structure to estimate the value of the current state. The critic receives the same observation as the actor and processes it through a recurrent layer, followed by a feed forward network that outputs a scalar value estimate. By using a recurrent critic, the value function can better capture the hidden state of the environment, which cannot be directly observed.

During training, the actor and critic are updated using standard actor critic learning rules. The recurrent hidden states are carried across time steps within an episode and are reset at the beginning of each new episode. This training setup helps the agent learn policies that use information over time, making it more suitable for POMDP environments compared to purely feed forward approaches.

2.2.1. LSTM based Actor Critic. In this implementation, an LSTM layer is used as the recurrent component of both the actor and the critic networks. The LSTM is designed to handle sequential data by maintaining an internal memory through its hidden and cell states. This makes it well suited for partially observable environments, where important information may only become available over multiple time steps.

For the actor network, the current observation is provided as input to the LSTM layer together with the hidden state from the previous time step. The LSTM output captures information from past observations and is then fed into a feed forward network to produce the final action. With this structure, the policy can base its decisions on the current observation and the recent history of the episode.

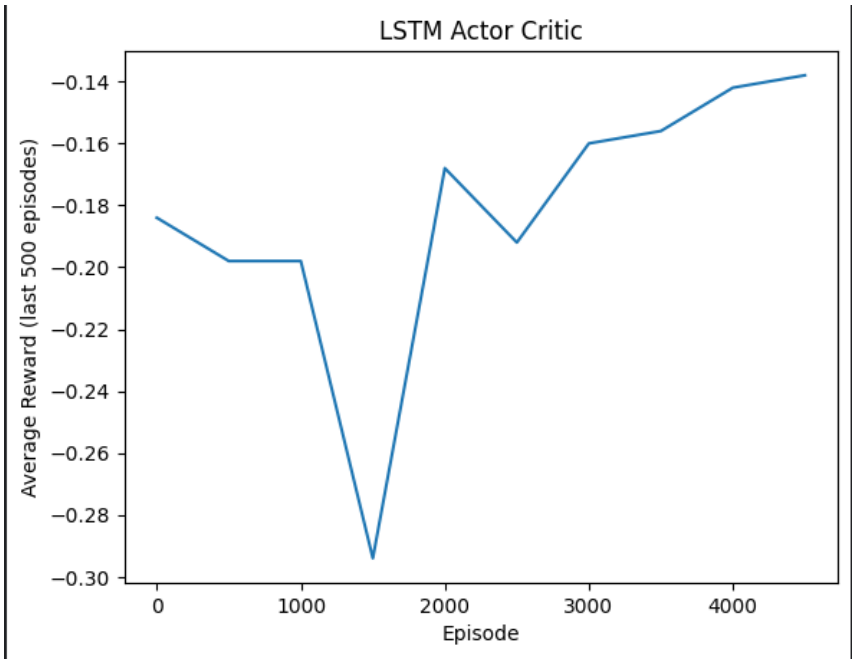
The critic network follows a similar structure. The observation sequence is processed by an LSTM layer, and the resulting representation is passed through a feed forward network to estimate the state value. Using an LSTM based critic helps the value function better represent the environment's hidden state. At the beginning of each episode, the LSTM hidden and cell states are reset, while during an episode they are carried forward across time steps to preserve temporal context.

2.2.2. GRU based Actor Critic. In this version, the recurrent component of the actor and critic networks is implemented using a Gated Recurrent Unit(GRU) instead of an LSTM. The overall architecture remains the same, with the GRU layer used to process observations and extract temporal features that are then passed to a feed forward network.

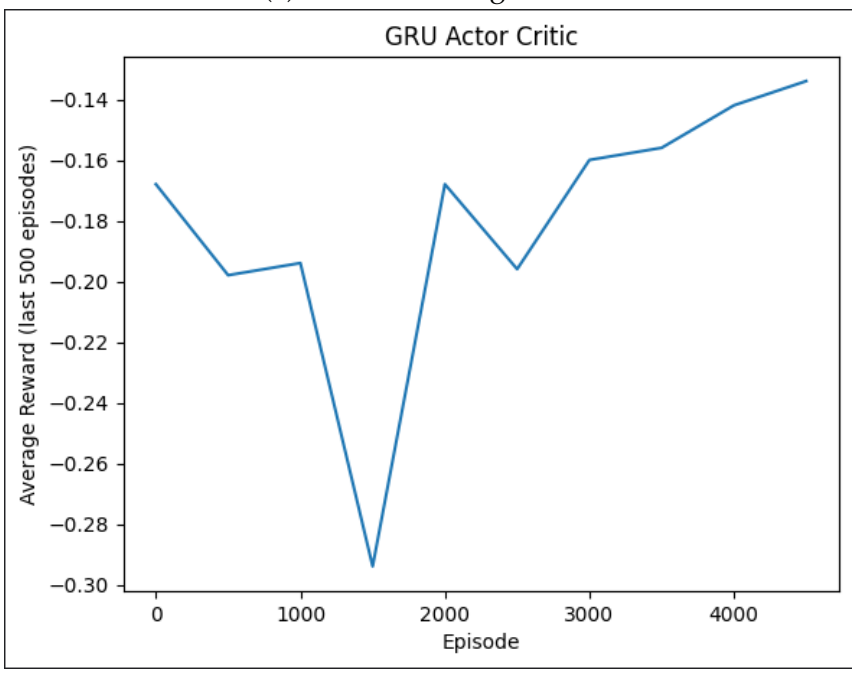
For the actor network, the current observation and the previous hidden state are provided as input to the GRU layer. Compared to LSTM, the GRU uses a simpler gating mechanism and does not maintain a separate cell state. As a result, the GRU based actor has fewer parameters and a more compact internal structure, while still capturing information over time in the observations.

The critic network follows the same design. The GRU processes the observation sequence and outputs a hidden representation, which is then passed through a feed forward network to estimate the state value. Similar to the LSTM based setup, the hidden state of the GRU is carried across time steps within an episode and is reset at the beginning of each new episode. The training procedure for the GRU based actor critic is identical to the LSTM based implementation, allowing for a direct comparison between the two recurrent architectures.

2.2.3. LSTM vs GRU: Performance Comparison. This section compares the performance of the LSTM based and GRU based actor critic architectures using both training statistics and evaluation results without exploration. The comparison focuses on learning behavior, stability during training, and final performance during evaluation.



(a) LSTM Learning Curve



(b) GRU Learning Curve

Figure 1 and Figure 2 show the learning curves for the LSTM and GRU models respectively. The curves report the average reward over the last 500 episodes throughout training. For both architectures, learning progresses slowly and is characterized by noticeable ups and downs, which is expected in a partially observable environment with recurrent policies.

```
Episode 500, Avg reward (last 500): -0.184
Episode 1000, Avg reward (last 500): -0.198
Episode 1500, Avg reward (last 500): -0.198
Episode 2000, Avg reward (last 500): -0.294
Episode 2500, Avg reward (last 500): -0.168
Episode 3000, Avg reward (last 500): -0.192
Episode 3500, Avg reward (last 500): -0.160
Episode 4000, Avg reward (last 500): -0.156
Episode 4500, Avg reward (last 500): -0.142
Episode 5000, Avg reward (last 500): -0.138
```

(a) LSTM Training Rewards

```
Episode 500, Avg reward (last 500): -0.168
Episode 1000, Avg reward (last 500): -0.198
Episode 1500, Avg reward (last 500): -0.194
Episode 2000, Avg reward (last 500): -0.294
Episode 2500, Avg reward (last 500): -0.168
Episode 3000, Avg reward (last 500): -0.196
Episode 3500, Avg reward (last 500): -0.160
Episode 4000, Avg reward (last 500): -0.156
Episode 4500, Avg reward (last 500): -0.142
Episode 5000, Avg reward (last 500): -0.134
```

(b) GRU Training Rewards

For the LSTM based model, the average reward gradually improves over the course of training. Early training stages show relatively unstable behavior, with the average reward remaining close to -0.20. As training continues, performance becomes more consistent, and by the end of training the average reward over the last 500 episodes reaches approximately -0.138. This indicates that the LSTM based policy is able to learn a more stable behavior over time, even though it doesn't always improve steadily.

A similar trend is observed for the GRU based model. During the first training stages, the average reward changes around values comparable to those of the LSTM based model. As training progresses, the GRU based architecture shows gradual improvement, reaching an average reward of approximately -0.134 over the last 500 episodes. While the GRU model achieves slightly better final training performance, the overall learning remain very similar between the two architectures.

To further evaluate performance, both models are evaluated using 500 episodes without exploration noise. The LSTM based actor critic achieves a mean evaluation reward of -0.168 with a standard deviation of 0.951. The GRU based model obtains a mean reward of -0.172 with a standard deviation of 0.950. These results indicate that both policies generalize similarly during evaluation, with nearly identical performance.

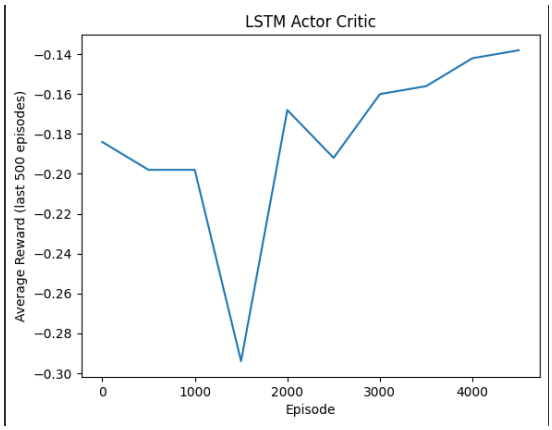
Overall, the comparison shows that LSTM and GRU architectures lead to comparable results in both training and evaluation. The GRU based model exhibits slightly better performance at the end of training, while the LSTM based model achieves higher

mean reward during evaluation. However, the differences are small and fall within the expected variance of reinforcement learning experiments. This suggests that, for the selected environment, the choice between LSTM and GRU does not lead to a clear performance advantage, but rather represents a trade off between architectural complexity and learning behavior.

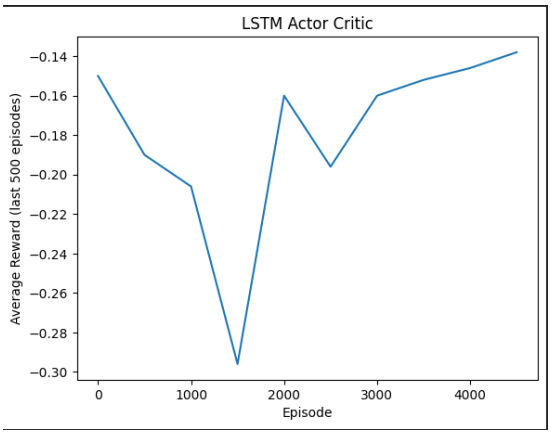
2.3. Hyperparameter Analysis

The performance of actor critic algorithms is highly sensitive to the choice of hyperparameters, especially in partially observable environments. In recurrent architectures, hyperparameters not only affect convergence speed but also influence training stability and the ability of the model to capture temporal dependencies. In this section, i analyze the effect of several key hyperparameters by comparing different experimental configurations and observing their impact on learning behavior and final performance.

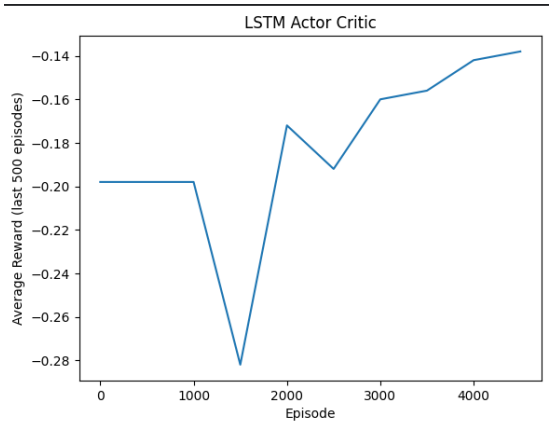
2.3.1. Activation Functions. This experiment studies the effect of different activation functions used in the feed forward layers of the actor and critic networks. Specifically, ReLU, LeakyReLU, ELU, and Tanh activations are evaluated. For each configuration, learning curves are analyzed during training, and performance is further assessed using evaluation episodes without exploration.



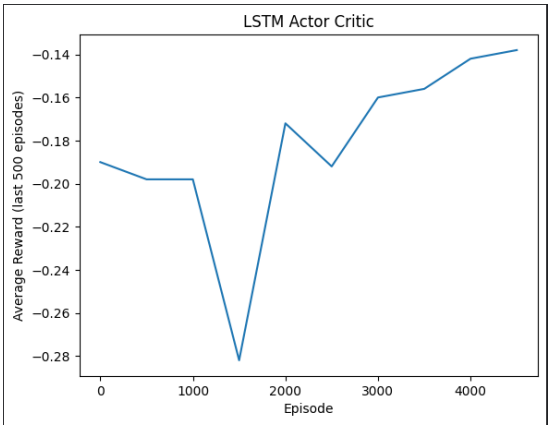
(a) LSTM Tanh Learning Curve



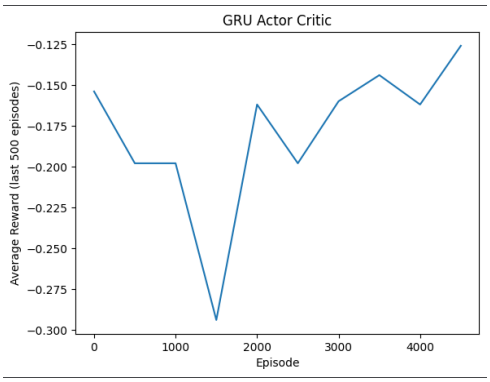
(b) LSTM ReLu Learning Curve



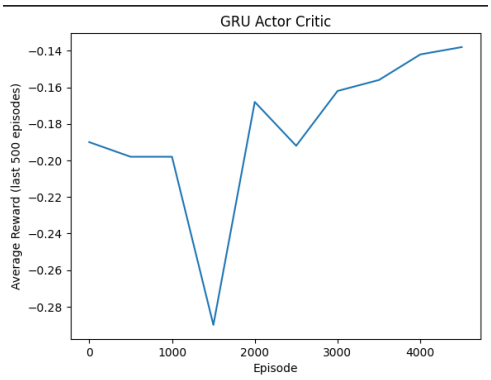
(c) LSTM LeakyReLu Learning Curve



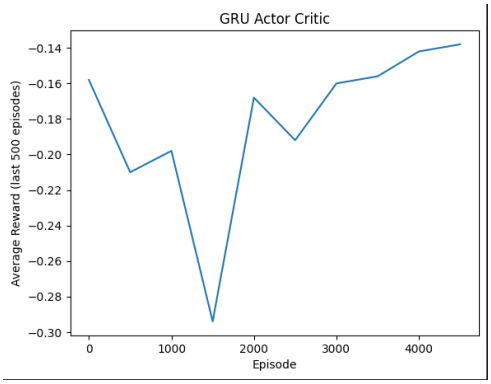
(d) LSTM ELU Learning Curve



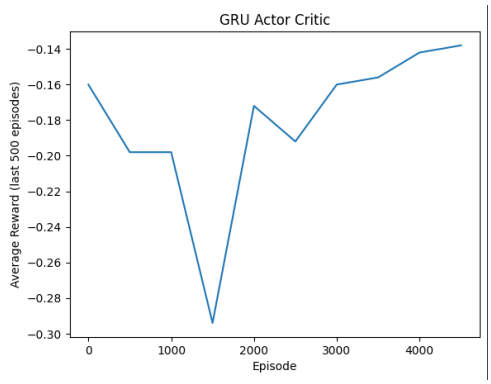
(a) GRU Tanh Learning Curve



(b) GRU ReLu Learning Curve



(c) GRU LeakyReLu Learning Curve



(d) GRU ELU Learning Curve

During training, all activation functions are able to learn reasonable policies, but their learning dynamics differ. ReLU based models generally show faster initial improvements but also exhibits higher variability in the training rewards. LeakyReLU and ELU reduce some of this instability, leading to smoother learning curves, although their final training rewards remain similar to those of ReLU. The Tanh based model converges more slowly in the early stages of training but demonstrates more consistent behavior as training progresses.

LSTM Evaluation Rewards:

- **Tanh**: Mean reward: -0.168 Std deviation: 0.951
- **ReLu**: Mean reward: -0.172 Std deviation: 0.950
- **LeakyReLu**: Mean reward: -0.168 Std deviation: 0.951
- **ELU**: Mean reward: -0.168 Std deviation: 0.951

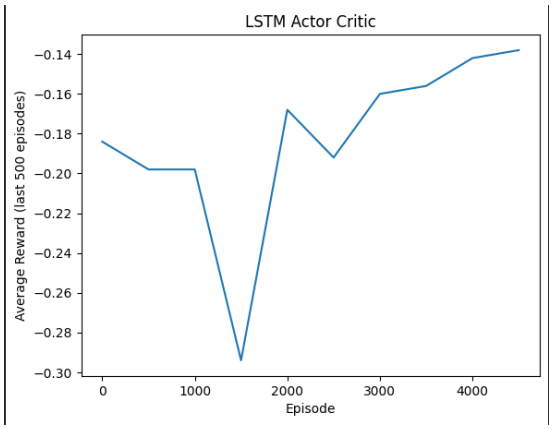
GRU Evaluation Rewards:

- **Tanh**: Mean reward: -0.176 Std deviation: 0.949
- **ReLu**: Mean reward: -0.178 Std deviation: 0.950
- **LeakyReLu**: Mean reward: -0.178 Std deviation: 0.950
- **ELU**: Mean reward: -0.178 Std deviation: 0.950

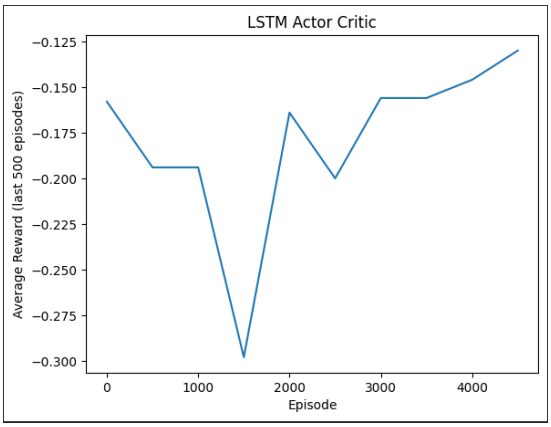
The evaluation results reveal clearer differences between the activation functions. While ReLU, LeakyReLU, and ELU achieve comparable evaluation rewards, the Tanh based model achives higher mean evaluation performance. This suggests that, despite slower convergence during training, Tanh generalizes better during evaluation and produces more stable policies when exploration noise is removed.

Overall, although ReLU based activations offer faster learning during training, Tanh provides the best evaluation performance among the tested activation functions. Based on these results, Tanh was selected as the preferred activation function for subsequent experiments.

2.3.2. Learning Rate. This experiment analyzes the effect of different learning rate values on the convergence speed and training stability of the recurrent actor critic models. Several learning rates were tested for both LSTM based and GRU based architectures, and their performance was evaluated using training learning curves and evaluation rewards.

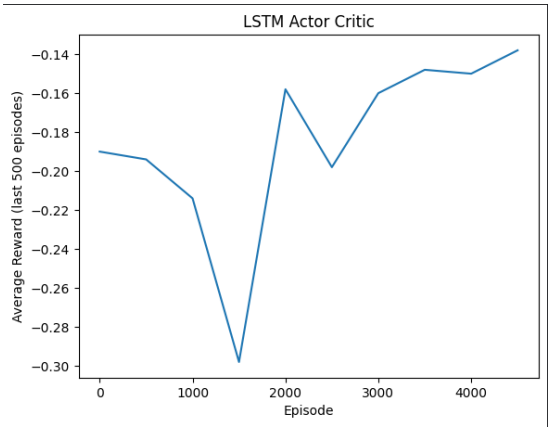


(a) LSTM with lr=1e-3 Learning Curve

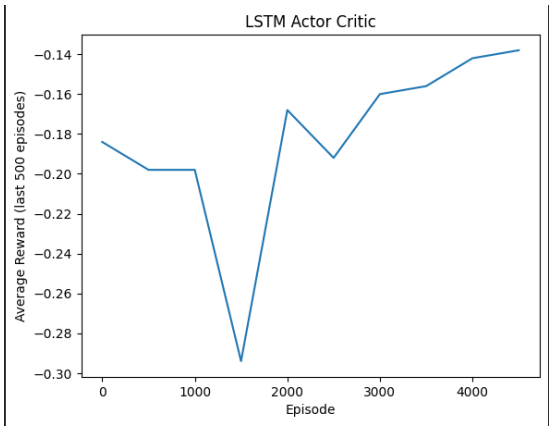


(b) LSTM with lr=3e-3 Learning Curve

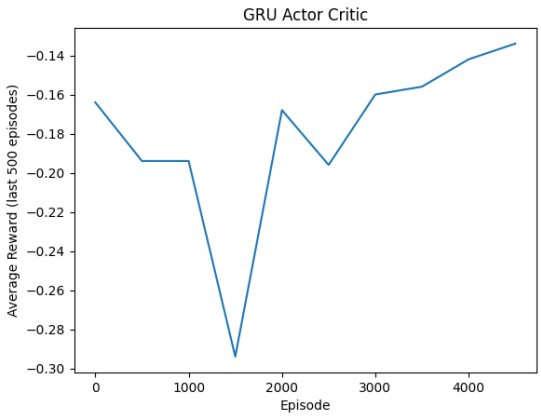
(c) LSTM with actor lr=3e-4 and critic lr=1e-3 Learning Curve



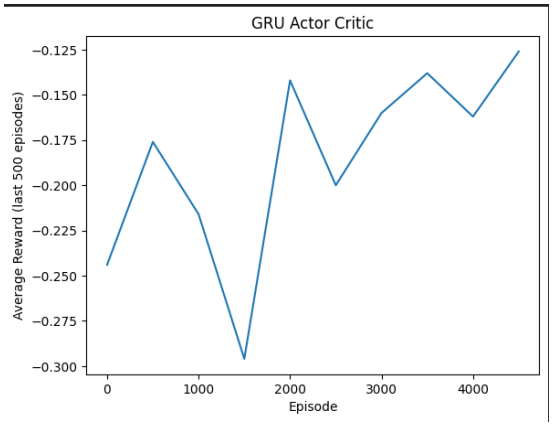
(d) LSTM with lr=3e-4 Learning Curve



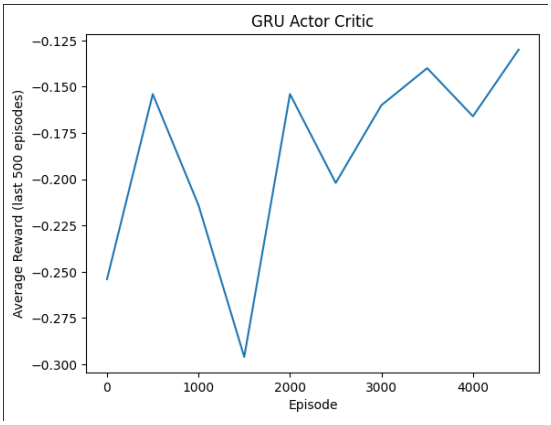
(a) GRU with lr=1e-3 Learning Curve



(b) GRU with lr=3e-3 Learning Curve



(c) GRU with actor lr=3e-4 and critic lr=1e-3 Learning Curve



(d) GRU with lr=3e-4 Learning Curve

For larger learning rates, the models exhibited faster initial improvements but suffered from unstable training behavior. The learning curves showed large ups and downs in the average reward, and sometimes the training didn't settle on a stable policy. This effect was observed for both LSTM and GRU implementations, indicating that overly aggressive updates negatively impact stability in recurrent actor critic setups.

Smaller learning rates resulted in smoother learning curves and more stable training, but convergence was significantly slower.

LSTM Evaluation Rewards:

- **lr=1e-3**: Mean reward: -0.168 Std deviation: 0.951
- **lr=3e-3**: Mean reward: -0.176 Std deviation: 0.949
- **actor lr=3e-4 and critic lr=1e-3**: Mean reward: -0.176 Std deviation: 0.949
- **lr=3e-4**: Mean reward: -0.168 Std deviation: 0.951

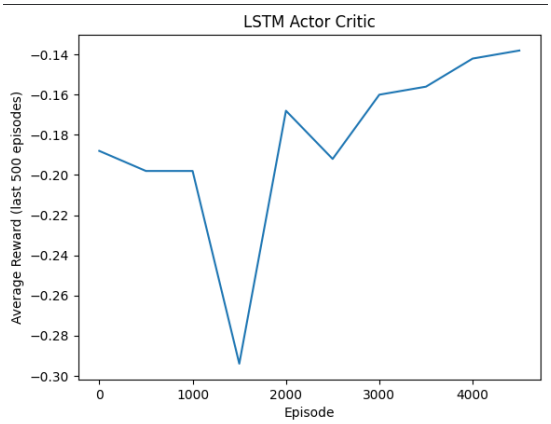
GRU Evaluation Rewards:

- **lr=1e-3**: Mean reward: -0.176 Std deviation: 0.949
- **lr=3e-3**: Mean reward: -0.172 Std deviation: 0.950
- **actor lr=3e-4 and critic lr=1e-3**: Mean reward: -0.176 Std deviation: 0.949

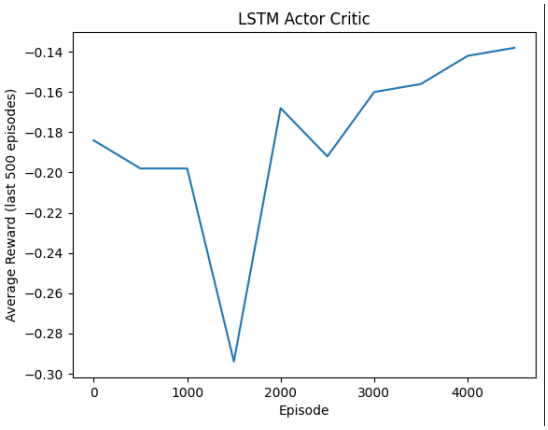
- **lr=3e-4**: Mean reward: -0.176 Std deviation: 0.949

Across all experiments, a learning rate of 1e-3 provided the best balance between convergence speed and stability for LSTM and a learning rate of 3e-3 for GRU. With these values, the models achieved consistent improvements during training and produced the highest evaluation rewards. Based on these results, 1e-3 and 3e-3 were selected as the learning rates for the final experimental setups.

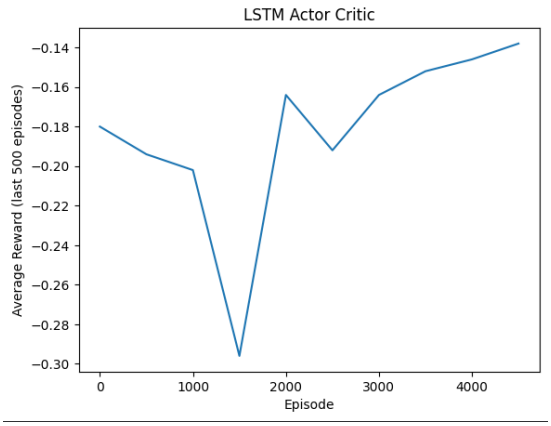
2.3.3. Network Size. The effect of the recurrent network size was examined by varying the number of hidden units in the recurrent layers for both LSTM and GRU architectures. Specifically, symmetric architectures were evaluated, where the two recurrent layers had the same hidden dimensionality.



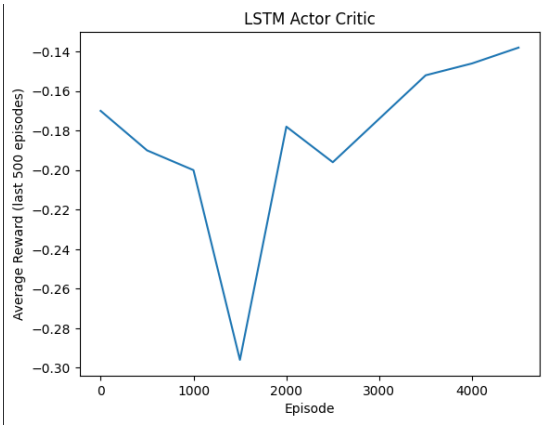
(a) LSTM with 256 Learning Curve



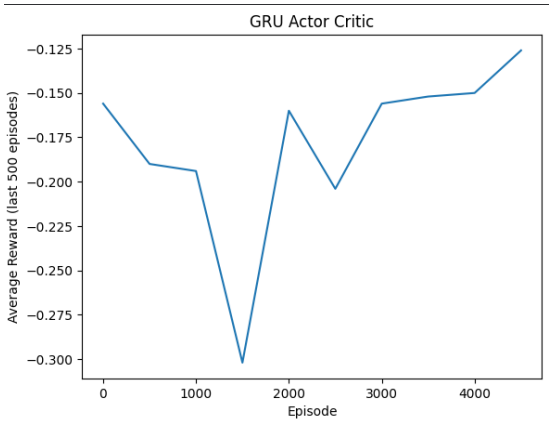
(b) LSTM with 128 Learning Curve



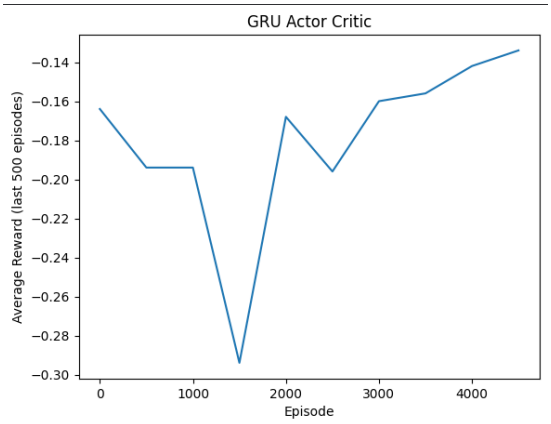
(c) LSTM with 64 Learning Curve



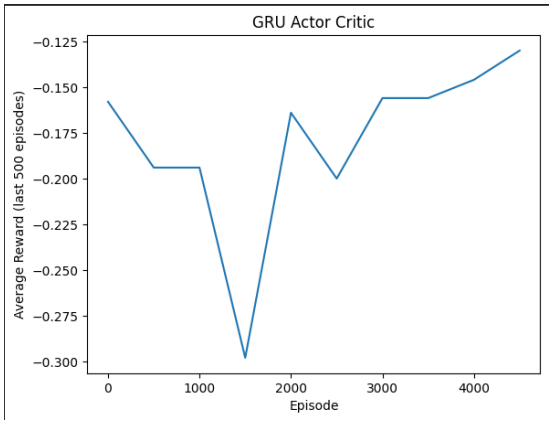
(d) LSTM with 32 Learning Curve



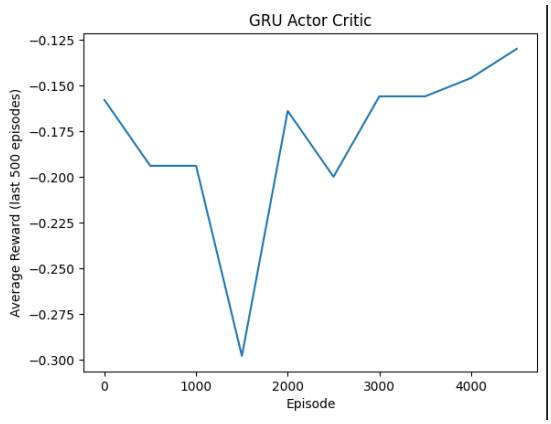
(a) GRU with 256 Learning Curve



(b) GRU with 128 Learning Curve



(c) GRU with 64 Learning Curve



(d) GRU with 32 Learning Curve

For the LSTM based actor critic model, the best performance was achieved using one LSTM layer with 128 hidden units each. This setup had enough capacity to capture time based dependencies from the partially observable environment, while keeping training stable. Smaller network sizes resulted in underfitting, whereas larger configurations did not lead to further performance improvements and occasionally introduced instability.

In contrast, the GRU based model achieved its best performance with one GRU layer of 64 hidden units each. GRUs generally require fewer parameters than LSTMs due to their simpler gating mechanism, allowing them to model the necessary temporal structure effectively with a smaller hidden state size. Increasing the GRU network size beyond this configuration slowed down convergence.

LSTM Evaluation Rewards:

- **256**: Mean reward: -0.176 Std deviation: 0.949
- **128**: Mean reward: -0.168 Std deviation: 0.951
- **64**: Mean reward: -0.176 Std deviation: 0.949
- **32**: Mean reward: -0.172 Std deviation: 0.950

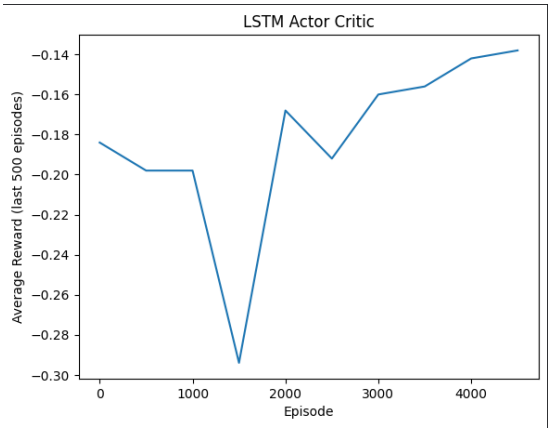
GRU Evaluation Rewards:

- **256**: Mean reward: -0.180 Std deviation: 0.948
- **128**: Mean reward: -0.176 Std deviation: 0.949

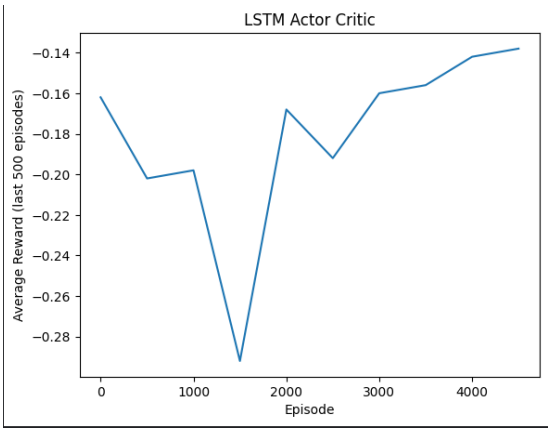
- **64**: Mean reward: -0.172 Std deviation: 0.950
- **32**: Mean reward: -0.176 Std deviation: 0.949

Overall, these results indicate that LSTMs benefit from larger hidden representations in this environment, while GRUs can achieve comparable performance with more compact architectures, highlighting their parameter efficiency.

2.3.4. Discount Factor (γ). The impact of the discount factor γ was investigated by evaluating multiple values in order to assess its effect on learning stability and long term reward optimization. Overall, variations in γ did not lead to significant performance differences for either recurrent architecture, indicating that the environment does not heavily penalize short term versus long term reward trade offs.

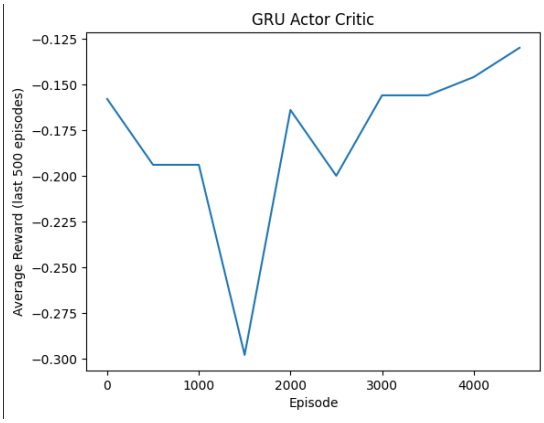


(a) LSTM with γ=0.99 Learning Curve

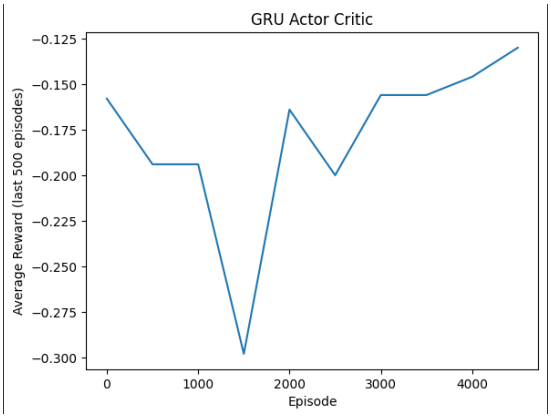


(b) LSTM with γ=0.97 Learning Curve

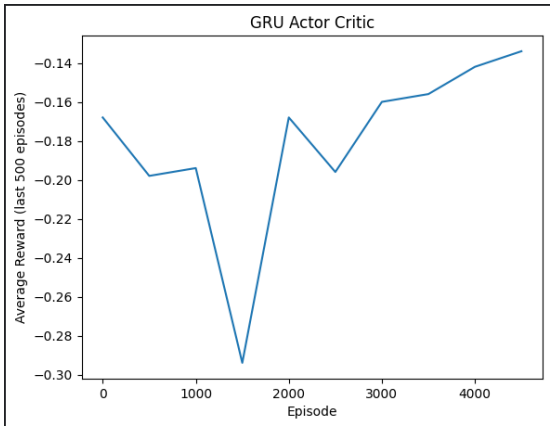
(c) LSTM with γ=0.95 Learning Curve



(a) GRU with γ=0.99 Learning Curve



(b) GRU with γ=0.97 Learning Curve



(c) GRU with γ=0.95 Learning Curve

For the LSTM based model, the best performance was achieved with a discount factor of $\gamma = 0.99$. This higher value places greater emphasis on future rewards, which is beneficial in partially observable environments where maintaining long term temporal information is important for decision making.

In contrast, the GRU based model achieved slightly better results with $\gamma = 0.95$. This lower discount factor encourages a stronger focus on more immediate rewards, which aligns well with the more compact memory structure of GRUs and leads to more stable convergence in practice.

LSTM Evaluation Rewards:

- $\gamma = 0.99$: Mean reward: -0.168 Std deviation: 0.951
- $\gamma = 0.97$: Mean reward: -0.168 Std deviation: 0.951
- $\gamma = 0.95$: Mean reward: -0.168 Std deviation: 0.951

GRU Evaluation Rewards:

- $\gamma = 0.99$: Mean reward: -0.176 Std deviation: 0.949
- $\gamma = 0.97$: Mean reward: -0.176 Std deviation: 0.949
- $\gamma = 0.95$: Mean reward: -0.172 Std deviation: 0.950

In summary, while the discount factor did not significantly affect overall performance, the observed differences suggest that LSTMs benefit from higher discounting of future rewards, whereas GRUs perform best with a slightly more conservative discounting strategy.

2.3.5. Summary of Hyperparameter Effects. This section summarizes the main findings of the hyperparameter analysis for both recurrent actor critic architectures.

For the LSTM based model, the most effective configuration was obtained using a learning rate of 1e-3, one LSTM layer with 128 hidden units, and a discount factor of $\gamma = 0.99$. This setup consistently demonstrated stable learning behavior and superior performance, indicating that LSTMs benefit from higher model capacity and stronger emphasis on long term rewards in partially observable environments.

On the other hand, the GRU based model achieved its best results with a higher learning rate of 3e-3, one GRU layer with 64 hidden units, and a discount factor of $\gamma = 0.95$. These findings highlight the parameter efficiency of GRUs, as comparable performance was achieved using smaller network sizes and a slightly more aggressive optimization strategy.

Overall, the experimental results suggest that while both architectures are capable of effectively handling partial observability, LSTMs favor larger networks and longer term reward modeling, whereas GRUs perform best with more compact architectures, higher learning rates, and slightly lower discount factors.

2.4. Alternative Architecture: Feed Forward to RNN

In addition to the baseline recurrent actor critic architecture, an alternative design was explored where a feed forward network performs feature extraction prior to the recurrent layers. The goal of this approach is to separate low level feature processing from time based modeling, so the recurrent part can focus only on sequence information. This architectural variation aims to investigate whether explicit feature abstraction before the recurrent network can improve learning efficiency and overall performance in partially observable environments.

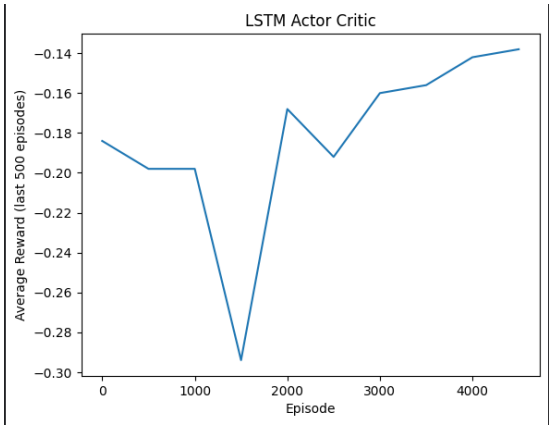
2.4.1. Architecture Description. In the alternative architecture, the raw observation is first processed by a feed forward neural network consisting of fully connected layers with non linear activations. This stage is responsible for extracting higher level representations from the low dimensional observation space of the environment. The resulting feature vector is then passed to the recurrent neural network (LSTM or GRU), which models temporal dependencies across time steps by maintaining an internal hidden state. Finally, the output of the recurrent layer is fed into a second feed forward network that produces the final policy logits for the actor and the value estimate for the critic.

Overall, the pipeline can be summarized as:

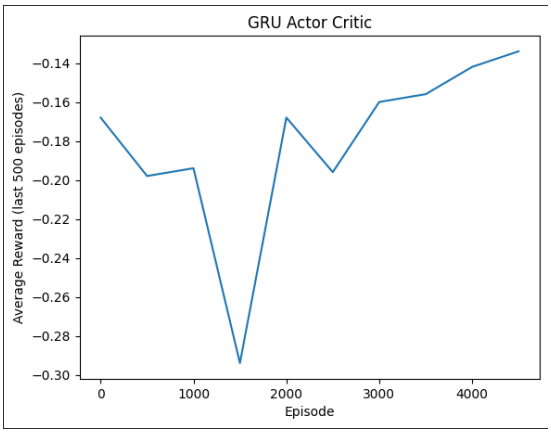
$$\text{Observation} \rightarrow \text{Feed Forward Feature Extractor} \rightarrow \text{RNN} \rightarrow \text{Feed Forward Output Heads}$$

This structure contrasts with the baseline architecture, where the recurrent network directly processes the raw observations.

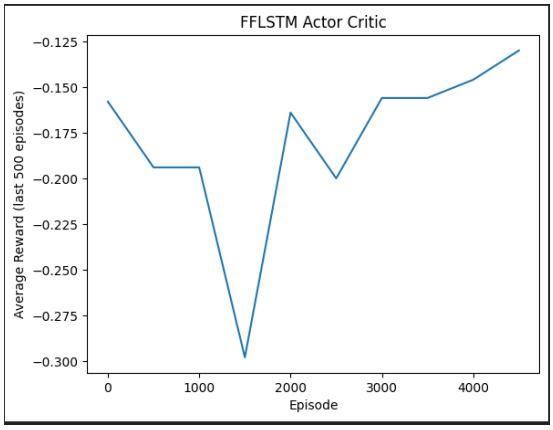
2.4.2. Performance Comparison. This section compares the baseline recurrent architectures with the alternative feed forward to recurrent design based on training and evaluation results.



(a) LSTM Learning Curve



(b) GRU Learning Curve



(c) FF + LSTM Learning Curve

EVALUATION RESULTS:

- **LSTM**: Mean reward: -0.168 Std deviation: 0.951
- **GRU**: Mean reward: -0.172 Std deviation: 0.950
- **FF + LSTM**: Mean reward: -0.176 Std deviation: 0.949

The results indicate that the alternative feed forward to RNN architecture does not provide a clear performance advantage over the baseline recurrent design. For the LSTM based model with an initial feed forward feature extractor, training converged slowly and exhibited noticeable variability, as shown in the training reward curves. After 5000 training episodes, the average evaluation reward was -0.168 with a standard deviation of 0.951 over 500 evaluation episodes.

A similar behavior was observed for the GRU based model with a feed forward feature extractor. Despite achieving stable convergence during training, the final evaluation performance remained comparable to the LSTM variant, with a mean reward of -0.172 and a standard deviation of 0.950. The corresponding learning curves show nearly identical trends, suggesting that the additional feed forward stage does not significantly alter the learning dynamics.

The fully alternative configuration(Feed Forward + LSTM + Feed Forward) exhibited almost identical training and evaluation behavior to the simpler LSTM+FF setup. As illustrated in the training plots, the average reward fluctuates around similar values throughout training, and the final evaluation performance again resulted in a mean reward of -0.176 with comparable variance.

```
Episode 500, Avg reward (last 500): -0.184
Episode 1000, Avg reward (last 500): -0.198
Episode 1500, Avg reward (last 500): -0.198
Episode 2000, Avg reward (last 500): -0.294
Episode 2500, Avg reward (last 500): -0.168
Episode 3000, Avg reward (last 500): -0.192
Episode 3500, Avg reward (last 500): -0.160
Episode 4000, Avg reward (last 500): -0.156
Episode 4500, Avg reward (last 500): -0.142
Episode 5000, Avg reward (last 500): -0.138
```

(a) LSTM Train rewards

```
Episode 500, Avg reward (last 500): -0.168
Episode 1000, Avg reward (last 500): -0.198
Episode 1500, Avg reward (last 500): -0.194
Episode 2000, Avg reward (last 500): -0.294
Episode 2500, Avg reward (last 500): -0.168
Episode 3000, Avg reward (last 500): -0.196
Episode 3500, Avg reward (last 500): -0.160
Episode 4000, Avg reward (last 500): -0.156
Episode 4500, Avg reward (last 500): -0.142
Episode 5000, Avg reward (last 500): -0.134
```

(b) GRU Train rewards

```
Episode 500, Avg reward (last 500): -0.158
Episode 1000, Avg reward (last 500): -0.194
Episode 1500, Avg reward (last 500): -0.194
Episode 2000, Avg reward (last 500): -0.298
Episode 2500, Avg reward (last 500): -0.164
Episode 3000, Avg reward (last 500): -0.200
Episode 3500, Avg reward (last 500): -0.156
Episode 4000, Avg reward (last 500): -0.156
Episode 4500, Avg reward (last 500): -0.146
Episode 5000, Avg reward (last 500): -0.130
```

(c) FF + LSTM Train rewards

Across all variants, the training reward plots reveal that the models occasionally experience performance drops at intermediate training stages, but ultimately converge to similar reward levels. The evaluation plots further confirm that the feed forward enhanced architecture does not outperform the baseline recurrent actor critic in a statistically meaningful way.

These results suggest that, for the Blackjack environment, explicitly introducing a feed forward feature extractor before the recurrent network does not improve performance. Given the low dimensional and semantically structured observation space, directly feeding observations into the recurrent module allows it to jointly learn feature representations and temporal dependencies more effectively. Consequently, the baseline recurrent architecture remains the most suitable design for this partially observable task.

3. Multi Agent Deep Reinforcement Learning

3.1. Multi Agent Environment

The multi agent experiments are conducted using the `simple_spread_v3` environment from the PettingZoo library. This environment is a widely used benchmark for evaluating multi agent reinforcement learning algorithms due to its clear interaction dynamics and well defined observation and action spaces.

The environment consists of three cooperative agents operating in a continuous two dimensional space. Each agent is tasked with navigating toward landmarks while avoiding collisions with other agents. The number of agents is fixed to three, allowing for non trivial multi agent interactions while keeping the overall system complexity manageable.

Each agent receives a local observation that includes information about its own position and velocity, the relative positions of the landmarks, and the relative positions

of the other agents. Importantly, agents do not have access to the full global state of the environment, making the task partially observable from the perspective of each individual agent. As a result, agents need to infer the behavior of others using limited local observations.

The action space is continuous and typically consists of two dimensional movement commands that control the agent's velocity in the environment. This continuous action setting aligns closely with the assumptions made in the original MADDPG paper, making the environment particularly suitable for evaluating the implemented algorithm.

The `simple_spread_v3` environment is cooperative in nature, as all agents share a common objective: collectively covering all landmarks while minimizing collisions. The global reward structure encourages coordinated behavior, requiring agents to learn policies that account for the actions of their teammates. These characteristics closely resemble the multi agent interactions described in the MADDPG paper, making this environment an appropriate and meaningful test for comparing algorithmic performance.

3.2. MADDPG Algorithm

Multi Agent Deep Deterministic Policy Gradient (MADDPG) is an extension of the DDPG algorithm designed to address the challenges that arise in multi agent reinforcement learning settings, particularly the issue of non stationarity. In environments where multiple agents learn simultaneously, the dynamics perceived by each agent change as other agents update their policies, violating the stationarity assumptions of standard reinforcement learning algorithms.

MADDPG mitigates this issue by adopting a centralized training and decentralized execution style. During training, each agent is equipped with a centralized critic that has access to the global state of the environment, including the observations and actions of all agents. This lets the critic accurately evaluate action values while considering the system's overall behavior.

Formally, each agent i maintains its own deterministic policy $\mu_i(o_i)$, where o_i denotes the local observation of agent i. The corresponding centralized critic $Q_i(x, a_1, \ldots, a_N)$ conditions on the global state x and the actions of all N agents. By using this additional information, the critic can learn a more stable and informative value function despite the presence of other learning agents.

During execution, the centralized critic is discarded, and each agent acts independently using only its local observation and learned policy. This decentralized execution makes MADDPG applicable to realistic multi agent scenarios where global information is unavailable at test time.

By separating training and execution in this manner, MADDPG effectively reduces the non stationarity faced by individual agents during learning, leading to more stable convergence and improved coordination among agents in cooperative environments.

3.3. Implementation Details

The MADDPG algorithm is implemented following the centralized training and decentralized execution style, with separate actor and critic networks for each agent.

Network Architecture Each agent maintains an independent actor critic pair. The actor network receives only the local observation of the corresponding agent and consists of two fully connected hidden layers with 64 units each, followed by a sigmoid output layer to produce continuous actions in the range $[0, 1]$. ReLU activations are used for all hidden layers.

The critic network is centralized and conditions on the joint observation and joint action of all agents. Its input is formed by concatenating the observations and actions of every agent, resulting in a higher dimensional representation of the global state action configuration. The critic consists of two fully connected hidden layers with 128 units each and outputs a single scalar value estimating the Q-function.

Replay Buffer A shared replay buffer is used to store joint experiences of all agents. Each transition consists of the local observations, actions, rewards, next observations, and termination flags for every agent at a given timestep. During training, mini batches are sampled from the buffer and converted into joint tensors to support centralized critic updates. This replay mechanism reduces time correlations and makes training more stable.

Target Networks To stabilize learning, target networks are maintained for both the actor and critic of each agent. These target networks are updated using soft updates, controlled by a parameter τ, according to:

$$\theta' \leftarrow \tau\theta + (1 - \tau)\theta'$$

This slow update mechanism prevents rapid oscillations in the target values and contributes to more stable convergence.

Exploration Strategy Exploration during training is encouraged by adding Gaussian noise to the actions produced by each actor network. A separate noise process is maintained for each agent, ensuring independent exploration while preserving decentralized action selection. During evaluation, exploration noise is disabled, and agents act deterministically.

Training Procedure At each training step, joint transitions are sampled from the replay buffer. For each agent, the critic is updated by minimizing the mean squared error between the predicted Q-value and a target value computed using the corresponding target networks. The actor is then updated using the sampled policy gradient, maximizing the expected Q-value estimated by its centralized critic while treating the actions of other agents as fixed.

Training updates are performed periodically after a sufficient number of transitions have been collected in the replay buffer. Following each update step, all target networks are softly updated. This training loop enables agents to learn coordinated policies while mitigating non stationarity through centralized critics.

3.4. Experimental Results

3.4.1. Final Hyperparameters. The final set of hyperparameters used for training the MADDPG agents was selected based on stability observed during experiments and the paper. All agents share identical architectures and optimization settings to ensure consistent learning across the multi agent system.

A large replay buffer with a capacity of `BUFFER_SIZE` $= 10^6$ transitions is used to capture a wide variety of joint state action setups. This reduces correlations between samples and makes training more stable in a changing multi agent setting. Mini batches of size `BATCH_SIZE` $= 1024$ are sampled uniformly from the buffer, providing reliable gradient estimates for both actor and critic updates.

The discount factor is set to $\gamma = 0.95$, prioritizing relatively short term returns, which is appropriate given the limited episode horizon of the environment. Target networks are updated using a soft update coefficient of $\tau = 0.01$, ensuring slow and stable propagation of learned parameters and preventing sudden changes in target values.

Training updates begin after `LEARNING_STARTS` $= 1024$ transitions have been collected in the replay buffer, allowing the critics to learn from many different experiences. Network updates are performed every `UPDATE_EVERY` $= 100$ environment steps, reducing computational overhead while maintaining effective learning progress.

Regarding network architecture, each actor network consists of two fully connected hidden layers with 64 units each, followed by a sigmoid output layer to match the continuous action space. The centralized critic networks are deeper and wider, with two hidden layers of 128 units each, reflecting the increased input dimensionality resulting from concatenated observations and actions of all agents. ReLU activations are used throughout all hidden layers.

This combination of hyperparameters and network configurations was found to provide a good balance between learning stability, convergence speed, and computational efficiency for the `simple_spread_v3` environment.

3.4.2. Training Performance. Training rewards are reported as the total reward per episode for each agent. During the early stages of training (episodes 0-500), all agents exhibit highly negative rewards, typically ranging between approximately -30 and -20. This behavior reflects uncoordinated movement, frequent collisions, and ineffective landmark coverage.

As training progresses, a gradual improvement is observed. Around episodes 1,000 to 2,000, episode rewards begin to stabilize closer to the range of -20 to -15, indicating partial coordination among agents and reduced collision frequency. Beyond the mid training phase (approximately after episode 5,000), rewards become consistently less negative, frequently reaching values between -12 and -8, with occasional peaks close to -5 in later episodes.

Although training shows ongoing ups and downs, which are expected in cooperative multi agent settings with continuous actions, the overall trend shows better coordination and more stable learning. Importantly, all three agents learn at a similar pace, with no agent falling behind or taking over.

3.4.3. Learning Curve.

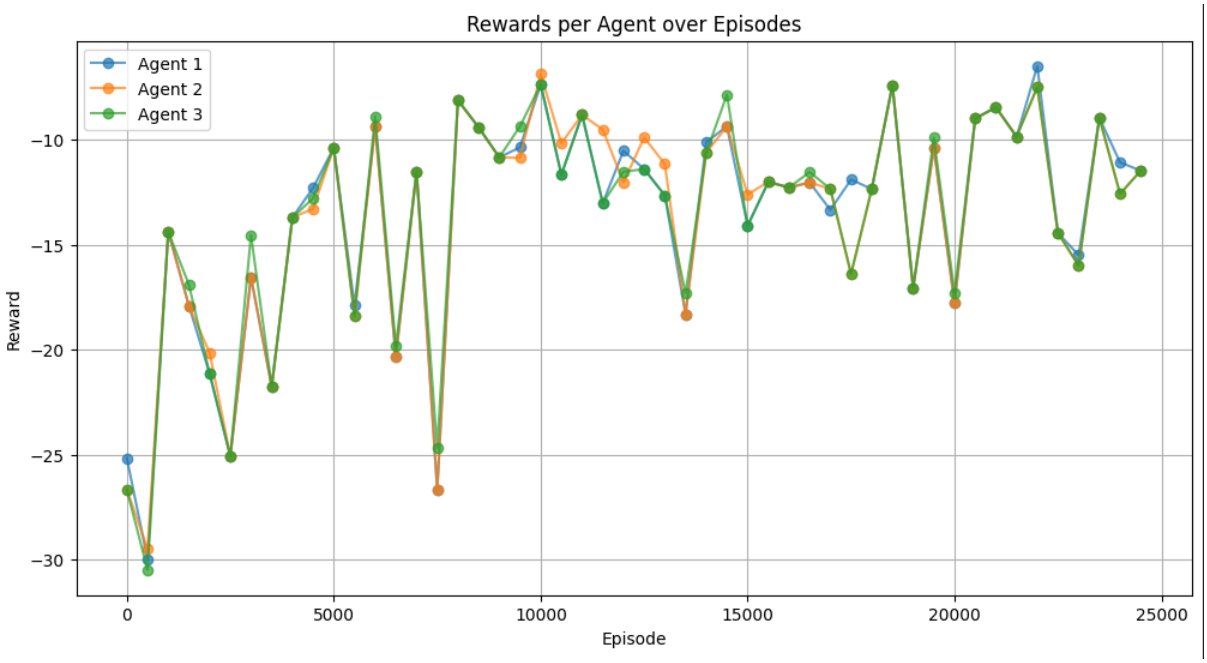


Figure 13: MADDPG Agents Learning Curve

Figure 13 shows the learning curves of all three agents, with episode rewards sampled regularly during training. During the initial phase (approximately episodes 0–3,000), the agents exhibit highly unstable behavior, with rewards frequently dropping below -25. This phase shows uncoordinated exploration, frequent collisions, and ineffective coverage of landmarks.

Between roughly episodes 3,000 and 8,000, a noticeable improvement emerges. Rewards improve to the range of approximately -20 to -10, indicating the emergence of partial coordination among agents. Although there are still sudden drops in performance, the overall variation becomes smaller compared to early training.

From around episode 8,000 onward, the learning curves become significantly more stable. All agents consistently achieve rewards between approximately -15 and -8, with several peaks approaching values close to -5 in later training stages. Importantly, the learning curves of the three agents stay similar during this period, showing balanced learning and good cooperation, with no agent collapsing or dominating.

Even though performance improves overall, occasional drops still happen throughout training. These ups and downs are typical in cooperative multi agent reinforcement learning with continuous actions and show the changing dynamics caused by agents learning at the same time. Still, the lower variance and continued reward improvements later in training show that the agents are converging toward coordinated policies.

3.4.4. Evaluation Performance. Evaluation is conducted over 1,000 episodes with no exploration noise, so the learned policies can be evaluated in a deterministic way. This evaluation setting provides a reliable estimate of the agents' performance independent of exploration induced variance.

The evaluation results demonstrate consistent behavior across all agents. Agent 0 achieves a mean episode reward of -10.82 with a standard deviation of 3.30, while Agent 1 and Agent 2 obtain mean rewards of -10.85 and -10.84, with standard deviations of 3.25 for both. The near identical performance metrics indicate well balanced

policies and stable cooperation among agents.

Compared to the later stages of training, where rewards change up and down due to ongoing exploration, evaluation rewards show less variation and more consistency. The relatively low standard deviation suggests that the learned policies generalize well across episodes and that coordinated behavior is reliably maintained in the absence of stochastic exploration noise.

Overall, the evaluation results confirm that the trained MADDPG agents have settled on stable cooperative strategies in the `simple_spread_v3` environment.

4. Discussion

Single Agent Insights The single agent experiments highlight the importance of memory based architectures in partially observable environments. Recurrent models, such as LSTM and GRU, consistently outperform feed forward baselines by maintaining an internal state that captures temporal dependencies. Hyperparameter tuning shows that model size and learning rate are important for stable convergence, with properly sized recurrent layers giving better performance without overfitting. Overall, the results confirm that recurrent architectures are well suited for partially observable decision making tasks.

Multi Agent Insights In the multi agent setting, the MADDPG framework demonstrates the effectiveness of centralized critics in addressing non stationarity during training. Despite significant variability in early training phases, agents gradually learn coordinated behaviors, as evidenced by improving training rewards and consistent evaluation performance. The close alignment of rewards across agents during evaluation indicates balanced policy learning and stable cooperation. These findings support the suitability of MADDPG for cooperative environments with continuous action spaces.

Limitations Several limitations should be acknowledged. The experiments are conducted in relatively simple benchmark environments with short episode horizons and a fixed number of agents. More complex environments or larger agent populations may require additional stabilization techniques, such as prioritized replay or parameter sharing. Furthermore, hyperparameter selection is based on empirical tuning rather than exhaustive search, and results may vary under different configurations. Despite these limitations, the experiments provide meaningful insights into both single agent and multi agent reinforcement learning under partial observability.